

Học Sâu (Deep Learning)

Chương 3: Giới thiệu TensorFlow, PyTorch, JAX và Keras

Đại học Đông Á

July 23, 2026

Nội dung chương

1. Lịch sử phát triển các Framework Học sâu
2. Phân tích Sức mạnh của từng Framework
3. Mối quan hệ giữa Keras và các Backend
4. Bài toán cốt lõi: Bộ Phân loại Tuyến tính
5. Ứng dụng nâng cao (Tổng quan kiến trúc)
6. Tổng kết

Sự tiến hóa của các Framework

- Kỷ nguyên trước 2009: Các nhà nghiên cứu phải tự viết tay logic tính toán đạo hàm (thường bằng C++). Việc lập trình GPU gần như bất khả thi.
- **Theano (2009)**: Framework đầu tiên hỗ trợ tự động tính đạo hàm (autograd) và tính toán GPU. Là "tổ tiên" của mọi framework hiện đại.
- **TensorFlow (Cuối 2015)**: Do Google phát triển, kế thừa Theano và hỗ trợ huấn luyện quy mô lớn. Keras nhanh chóng tích hợp TF.
- **PyTorch (Cuối 2016)**: Do Meta (Facebook) phát triển để đối trọng với TF, nổi bật với tính linh hoạt (Dynamic Computation Graph).
- **JAX (2018)**: Một hệ thống gọn nhẹ từ Google kết hợp Autograd và trình biên dịch XLA, tối ưu vô cùng mạnh mẽ.

Điểm nổi bật của TensorFlow

- **Hệ sinh thái Enterprise (Doanh nghiệp):** TensorFlow không chỉ là thư viện, nó là một hệ sinh thái khổng lồ (TFX, TF Serving, TF Lite, TensorFlow.js).
- **Kiến trúc Đồ thị (Graph Execution):** Khởi điểm sử dụng Static Computation Graph giúp biên dịch và tối ưu trước khi chạy cực kỳ hiệu quả, phù hợp cho triển khai thực tế.
- **Eager Execution:** Từ bản TF 2.0, TensorFlow đã hỗ trợ chạy trực tiếp từng dòng lệnh (Eager) để dễ debug và nghiên cứu hơn.
- **Mở rộng dễ dàng:** Hỗ trợ out-of-the-box cho các hệ thống phân tán lên tới hàng nghìn GPU/TPU.

Điểm nổi bật của PyTorch

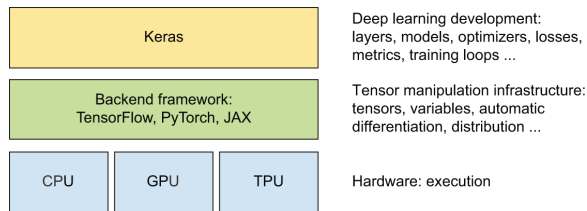
- **Thông trị mạng Nghiên cứu (Research):** Hơn 70% các bài báo khoa học AI mới đều sử dụng PyTorch.
- **Tính Pythonic cao:** Cú pháp rất gần gũi với Python bản địa (như Numpy), mang lại sự thoải mái cho lập trình viên.
- **Đồ thị tính toán Động (Dynamic Computation Graph):**
 - Đồ thị được xây dựng liên tục tại thời gian thực (on the fly).
 - Cực kỳ hữu ích để debug (có thể chèn print ở mọi nơi) và làm việc với độ dài chuỗi biến đổi (như NLP).

Điểm nổi bật của JAX

- **Siêu việt về Toán học:** JAX bản chất là thư viện *"Numpy trên GPU/TPU kết hợp với tự động đạo hàm (Autograd)"*.
- **Trình biên dịch XLA (Accelerated Linear Algebra):** Giúp tối ưu hóa toàn cục các chuỗi phép tính, tăng tốc độ xử lý phần cứng lên ngưỡng tối đa.
- **Lập trình hàm (Functional Programming):** Tránh sử dụng trạng thái toàn cục (global state), mọi hàm đều trong suốt (pure functions). Điều này giúp JAX cực kỳ an toàn khi chạy song song trên nhiều GPU/TPU.

High-level API và Low-level API

- **Low-level (TF, PyTorch, JAX):** Cung cấp các thao tác mảng tensor, tính toán đạo hàm tự động. Đòi hỏi code chi tiết, phức tạp.
- **High-level (Keras):** Cung cấp API trực quan (Layers, Models, Optimizers, Loss) để ráp nối nhanh chóng.
- Keras 3.0 trở lên: Có thể **chuyển đổi linh hoạt Backend** (TF $\leftrightarrow$ PyTorch $\leftrightarrow$ JAX) mà không cần viết lại code!



Hình 3.1: Cấu trúc đa Backend của Keras

Ôn tập toán học: Đường thẳng và Siêu phẳng

Mục tiêu: Xây dựng bộ phân loại tuyến tính (Linear Classifier) từ con số 0.

- Trong không gian 2D, một bộ phân loại tuyến tính là một **đường thẳng**.
- Phương trình dự đoán tổng quát:

$$y_pred = W \cdot x + b \quad (1)$$

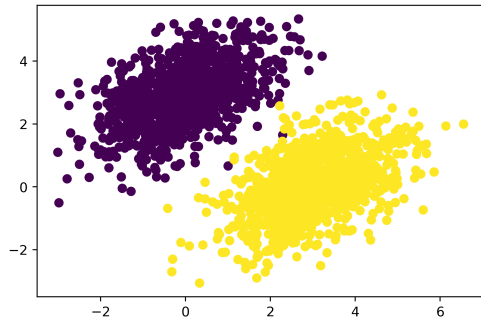
- W (Trọng số - Weights): Quyết định độ nghiêng/hướng của đường thẳng.
- b (Độ lệch - Bias): Quyết định vị trí tịnh tiến của đường thẳng so với gốc toạ độ.

Khởi tạo dữ liệu giả lập (Point Clouds)

Để minh họa, chúng ta tạo ra 2 cụm điểm (point clouds) phân bố ngẫu nhiên trong không gian 2D:

- **Cụm 1:** Ký hiệu màu nhạt, nhãn (label) = 0.
- **Cụm 2:** Ký hiệu màu đậm, nhãn (label) = 1.

Bài toán: Làm sao vẽ được một đường thẳng tách biệt chính xác 2 cụm điểm này bằng toán học thay vì đo đạc bằng mắt?



Hình 3.2: Hai cụm điểm dữ liệu ngẫu nhiên

Hàm mất mát (Loss) cho Bộ Phân Loại Tuyến Tính

- Để đo lường xem đường thẳng hiện tại "tốt" hay "tệ" so với dữ liệu thật, ta cần một độ đo chuẩn.
- Trong phân loại tuyến tính cơ bản, có thể dùng **MSE (Mean Squared Error - Sai số toàn phương trung bình)**:

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_pred_i - y_true_i)^2 \quad (2)$$

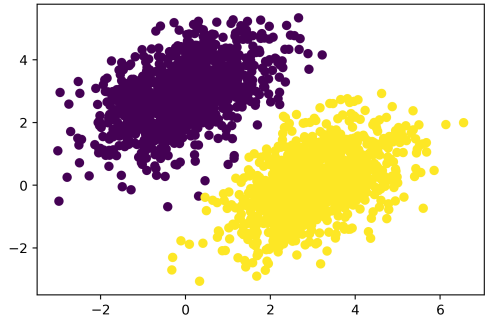
- Khi khoảng cách từ điểm đến đường thẳng lớn, hàm bình phương (x^2) sẽ phạt mô hình cực kỳ nặng, buộc W và b phải thay đổi để giảm khoảng cách này xuống.

Dự đoán ban đầu (Khởi tạo ngẫu nhiên)

Khi mô hình phân loại vừa được tạo ra:

- Trọng số W và b được khởi tạo bằng các giá trị ngẫu nhiên rất nhỏ (gần bằng 0).
- Kết quả: Đường thẳng phân chia không có bất kỳ ý nghĩa nào, dự đoán sai lệch, điểm MSE Loss rất cao.

Bước tiếp theo là sử dụng **Gradient Descent** để cập nhật.



Hình 3.3: Đường thẳng phân chia trước khi huấn luyện

Toán học của Cập nhật Gradient Descent

Để đường thẳng xoay hướng chuẩn, ta dùng công thức:

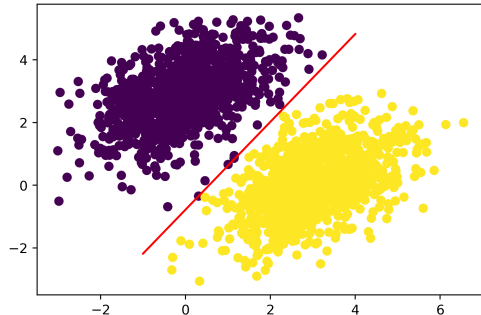
$$W \leftarrow W - \eta \cdot \nabla L(W) \quad (3)$$

- $\nabla L(W)$ là Gradient (đạo hàm) của hàm Loss theo W . Nó luôn chỉ về hướng làm cho sai số tăng lên nhanh nhất.
- Dấu trừ ($-$) đảm bảo ta đi "ngược" hướng Gradient để giảm sai số.
- η (Learning Rate - Tốc độ học): Siêu tham số quyết định mỗi bước thay đổi lớn cỡ nào. Nếu quá lớn: dao động mạnh, quá nhỏ: hội tụ chậm.

Kết quả sau Vòng lặp huấn luyện (Training Loop)

Sau nhiều bước lặp (epochs) cập nhật trọng số:

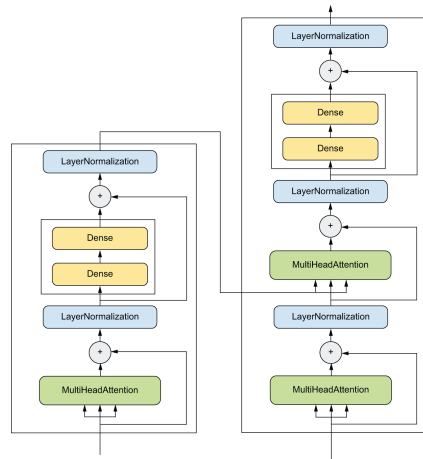
- Đường thẳng dần dịch chuyển tịnh tiến (b) và xoay độ nghiêng (W) cho đến khi cắt ngang khe hở giữa 2 cụm điểm.
- Lúc này, Loss hội tụ (đạt cực tiểu). Bộ phân loại tuyến tính đã học thành công!



Hình 3.4: Đường thẳng phân chia sau khi huấn luyện

Không chỉ dừng ở Mô hình Tuyến tính

- Mạng nơ-ron thực tế không chỉ có 1 đường thẳng, mà kết hợp hàng nghìn lớp biến đổi tuyến tính (Dense) đan xen với các hàm kích hoạt phi tuyến (ReLU, Softmax).
- Các kiến trúc mạng hiện đại có thể vô cùng phức tạp với hàng chục tỷ tham số, tiêu biểu như kiến trúc **Transformer** (Nền tảng của LLM).
- Keras giúp ta đóng gói các cụm toán học phức tạp thành từng Layer như chơi xếp hình.



Hình 3.5: Đồ thị kiến trúc của Transformer Block

- **TensorFlow, PyTorch, JAX:** Đóng vai trò là các Low-level Backend cung cấp động cơ tính toán ma trận và đạo hàm (Autograd) vô cùng mạnh mẽ trên GPU/TPU.
- **Keras:** Đóng vai trò là High-level API linh hoạt, tương thích đa backend, trực quan cho kỹ sư Học máy.
- **Bài toán cốt lõi:** Dù mạng đơn giản như Linear Classifier ($y = Wx + b$) hay phức tạp như Transformer, quy trình chuẩn vẫn là: Tính Loss $\rightarrow$ Tìm Gradient $\rightarrow$ Cập nhật W, b bằng Learning Rate.